ECEN 749 Lab 5 - Report

Yashas Bharadwaj
UIN: 635009332; Section: 601

Introduction

This lab focuses on developing device drivers within an embedded Linux environment. Device drivers are essential components of the operating system kernel, enabling user applications to communicate with hardware. This exercise was crucial for understanding the mechanisms of device drivers, particularly their role in facilitating access to and sharing of hardware resources controlled by the operating system. We built upon previous kernel module work to create a complete character device driver. Furthermore, we developed a corresponding Linux application to test this multiplication device driver, which, like earlier exercises, emphasized the practical application of this knowledge and reinforced fundamental skills in embedded systems development.

Procedure

I. Setup and Kernel Module Creation

1. Insert the USB drive containing the PetaLinux installation and project directory.
2. Run the necessary console commands to create a PetaLinux module named multiplier.

II. Character Device Driver Development

1. Develop a character device driver to manage read and write operations for the multiplication peripherals.
    - Use the provided sample files (my_chardev.c and my_chardev.h) as a reference.
    - The driver must be registered immediately after virtual memory mapping and unregistered before virtual memory unmapping.

III. Building and Loading the Module

1. Modify the multiplier.bb file to include all required header files.
2. Build the multiplier module using the command petalinux build.
3. Transfer the resulting kernel module, multiplier.ko, to the SD card.
4. Insert the prepared SD card into the Zybo board.
5. Use the picocom serial console to execute the manual's commands for mounting the SD card.
6. Load the kernel module onto the Zybo Linux system by running insmod multiplier.ko.
7. Verify the module's successful loading and output using the command dmesg | tail.

IV. Device Node Creation and User Application

1. Create the device node /dev/multiplier using the mknod command. (The successful creation was demonstrated to the TA.)

2. Write the user application, devtest, which uses the sample code from the lab manual to perform read and write operations on the device file /dev/multiplier.
3. Cross-compile the user application using the command specified in the lab manual.
4. Load the compiled user application onto the SD card.
5. Insert the SD card into the Zybo board and execute the user application. (The final outputs were demonstrated to the TA.)

Results

The multiplier device driver was successfully built and compiled on the CentOS workstations, and the resulting kernel module was loaded onto the Zybo Z7-10 board. A user application was also developed and built to generate inputs. This application used read and write operations to communicate with the device driver, passing inputs and receiving outputs. The 'multiplier' kernel character device driver facilitated interaction between the user application and the multiplier peripheral. Multiplication results were generated by the hardware peripheral, transferred to the kernel buffer by the device driver, and then passed back to the user application. Finally, the outputs were displayed on a serial console using the picocom application. A demonstration of the results was also given to the TA.

Conclusion

This lab exercise provided a significant enhancement to our understanding of device drivers within an embedded Linux environment. Through the exploration of kernel modules and their critical role in facilitating communication between user applications and hardware, we gained valuable insights into system-level operations. The practical experience of developing a character device driver, along with its accompanying Linux application, reinforced essential embedded systems development skills. This reinforced the practical importance of device drivers in enabling seamless hardware interaction. The knowledge acquired provides a solid foundation and the necessary tools for navigating and innovating within the embedded systems domain.

Questions

a) The ARM processor interacts with the multiplier block, which functions as an I/O device. The code for managing this interaction resides on the ARM processor and operates within the virtual memory space. To access the I/O memory, the kernel must invoke ioremap(). This function is essential as it populates the page table with the physical address entries of the device. A successful ioremap() call returns a kernel virtual address that maps to the beginning of the requested physical address range, enabling device access via the page tables.

b) No, I feel the multiplication operation would be quicker in the original Lab 3 setup. That's because the C code in Lab 3 talks straight to the hardware. This lab's C code, though, slows things down a bit because it has to talk with the hardware through the device

driver. Plus, if the operating system is busy with other stuff, that could add another little delay.

c)

Benefits of the Device Driver Approach in this Lab

- Higher Abstraction: Users are provided with a high level of abstraction, eliminating the need for knowledge about the underlying hardware and register interaction methods. They simply create an application file, provide inputs, and receive outputs, as the hardware's internal implementation is completely concealed.
- Hardware Independence/Flexibility: The implementation is not rigidly tied to specific hardware. It can be easily repurposed or extended to support additional hardware peripherals through code modifications.

Costs of the Device Driver Approach in this Lab

- Performance Overhead: The introduction of an extra layer of software leads to a longer execution time and performance overhead.
- Development Complexity: Developing a device driver is complex due to the necessity of ensuring compatibility with the specific hardware peripheral.
- Limited Visibility: The internal implementation is not visible to the user.

Analysis of Lab 3 Approach:

Benefits:

- Faster Execution/Lower Latency: Direct kernel interaction with the hardware peripheral eliminates the intermediate device driver layer, leading to quicker execution times.
- Increased Efficiency: The approach conserves hardware resources (e.g., CPU cycles and memory) by reducing overhead.

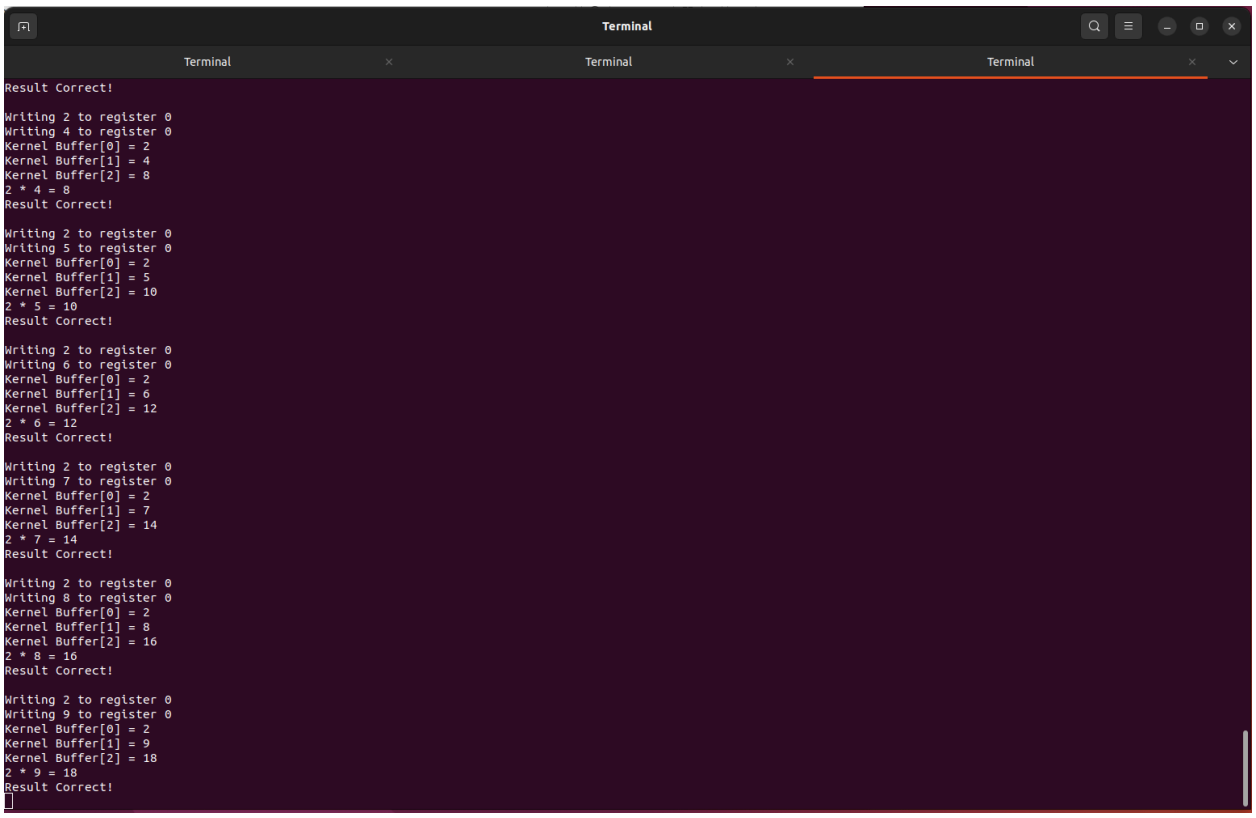
Costs:

- Hardware Expertise Required: Deep understanding of the peripheral's hardware design is necessary. In Lab 3, this involved generating the bitstream, loading it onto the FPGA board, and knowing the register space.
- Limited Extensibility: The implementation is highly specific to the targeted hardware peripheral, making it difficult to adapt or extend to support different or additional hardware.

d) Device registration is intentionally the final step in initialization to prevent premature kernel/application access. Early registration could lead to unexpected behavior if the kernel tries to access the module before setup (e.g., buffer allocation, address mapping) is complete. Registration is delayed until the device is fully ready for read/write operations. Conversely, unregistration is the first step in the exit routine, immediately halting kernel access as de-initialization begins.

Appendix

Picocom console:



```
Result Correct!

Writing 2 to register 0
Writing 4 to register 0
Kernel Buffer[0] = 2
Kernel Buffer[1] = 4
Kernel Buffer[2] = 8
2 * 4 = 8
Result Correct!

Writing 2 to register 0
Writing 5 to register 0
Kernel Buffer[0] = 2
Kernel Buffer[1] = 5
Kernel Buffer[2] = 10
2 * 5 = 10
Result Correct!

Writing 2 to register 0
Writing 6 to register 0
Kernel Buffer[0] = 2
Kernel Buffer[1] = 6
Kernel Buffer[2] = 12
2 * 6 = 12
Result Correct!

Writing 2 to register 0
Writing 7 to register 0
Kernel Buffer[0] = 2
Kernel Buffer[1] = 7
Kernel Buffer[2] = 14
2 * 7 = 14
Result Correct!

Writing 2 to register 0
Writing 8 to register 0
Kernel Buffer[0] = 2
Kernel Buffer[1] = 8
Kernel Buffer[2] = 16
2 * 8 = 16
Result Correct!

Writing 2 to register 0
Writing 9 to register 0
Kernel Buffer[0] = 2
Kernel Buffer[1] = 9
Kernel Buffer[2] = 18
2 * 9 = 18
Result Correct!
```

“Multiply.c” code:

```
#include <linux/module.h>  /* Needed by all modules */
#include <linux/moduleparam.h>  /* Needed for module parameters */
#include <linux/kernel.h>  /* Needed for printk and KERN_* */
#include <linux/init.h>       /* Need for __init macros  */
#include <linux/fs.h>       /* Provides file ops structure */
#include <linux/sched.h>   /* Provides access to the "current" process task structure */
```

```
#include <asm/uaccess.h>   /* Provides utilities to bring user space data into kernel space.
Note, it is processor arch specific. */
#include <linux/slab.h>       /* kmalloc and kfree definitions */
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <asm/io.h>
#include "xparameters.h"
#include <linux/ioport.h>   // IO memory allocation
#include "my_chardev_mem.h"
#include "xparameters_ps.h"

// From xparameters.h
#define PHY_ADDR    XPAR_MULTIPLY_0_S00_AXI_BASEADDR // physical address of
multiplier

// Size of physical address range for multiply
#define MEMSIZE        XPAR_MULTIPLY_0_S00_AXI_HIGHADDR -
XPAR_MULTIPLY_0_S00_AXI_BASEADDR + 1

/* Some defines */
#define DEVICE_NAME    "multiplier"
#define BUF_LEN        80

/* Function prototypes, so we can setup the function pointers for dev
   file access correctly. */
int init_module(void);
void cleanup_module(void);
static int device_open(struct inode *, struct file *);
static int device_close(struct inode *, struct file *);
static ssize_t device_read(struct file *, char *, size_t, loff_t *);
static ssize_t device_write(struct file *, const char *, size_t, loff_t *);

// Virtual address pointing to multiplier
void* virt_addr;

/*
 * Global variables are declared as static, so are global but only
 * accessible within the file.
 */
static int Major;        /* Major number assigned to our device driver */
static int Device_Open = 0;    /* Flag to signify open device */
```

```
/* This structure defines the function pointers to our functions for
   opening, closing, reading and writing the device file.  There are
   lots of other pointers in this structure which we are not using,
   see the whole definition in linux/fs.h */
static struct file_operations fops = {
  .read = device_read,
  .write = device_write,
  .open = device_open,
  .release = device_close
};


static int __init my_init(void)
{

  printk(KERN_INFO "Mapping virtual address... \n");

  // map virtual address to multiplier physical address
  virt_addr = ioremap(PHY_ADDR, MEMSIZE);

  printk(KERN_INFO "Physical Address = %p \n", (void *)PHY_ADDR);
  printk(KERN_INFO "Virtual Address = %p \n", virt_addr);

  /* This function call registers a device and returns a major number
     associated with it.  Be wary, the device file could be accessed
     as soon as you register it, make sure anything you need (ie
     buffers ect) are setup _BEFORE_ you register the device.*/
  Major = register_chrdev(0, DEVICE_NAME, &fops);


  /* Negative values indicate a problem */
  if (Major < 0) {
    /* Make sure you release any other resources you've already
       grabbed if you get here so you don't leave the kernel in a
       broken state. */
    printk(KERN_ALERT "Registering char device failed with %d\n", Major);
    return Major;
  }

  printk(KERN_INFO "Registered a device with dynamic Major number of %d\n", Major);
  printk(KERN_INFO "Create a device file for this device with this command:\n'mknod /dev/%s c
%d 0'.\n", DEVICE_NAME, Major);

  return 0;        /* success */
```

```
}


static void __exit my_exit(void)
{
    // Unregister the device
    unregister_chrdev(Major, DEVICE_NAME);

    printk(KERN_ALERT "unmapping virtual address space... \n");
    iounmap((void *)virt_addr);
}


void my_cleanup(void)
{
  /*
   * Unregister the device
   */
  unregister_chrdev(Major, DEVICE_NAME);

  kfree(msg_bf_Ptr);            /* free our memory (note the ordering
                                here) */

}


/*
 * Called when a process tries to open the device file, like "cat
 * /dev/my_chardev".  Link to this function placed in file operations
 * structure for our device file.
 */
static int device_open(struct inode *inode, struct file *file)
{


/* In these case we are only allowing one process to hold the device
     file open at a time. */
  if (Device_Open)        /* Device_Open is my flag for the
                  usage of the device file (definied
                  in my_chardev.h)  */
    return -EBUSY;        /* Failure to open device is given
                  back to the userland program. */

  Device_Open++;        /* Keeping the count of the device
```

```
                opens. */


    try_module_get(THIS_MODULE);    /* increment the module use count
                (make sure this is accurate or you
                won't be able to remove the module
                later. */

     cur_Ptr = msg_bf_Ptr;         /* set the ptr to the beginning of the
                              message */

    printk("Device opened successfully \n");
    return 0;
}


/*
 * Called when a process closes the device file.
 */
static int device_close(struct inode *inode, struct file *file)
{

    Device_Open--;        /* We're now ready for our next
                caller */

  /*
   * Decrement the usage count, or else once you opened the file,
   * you'll never get get rid of the module.
   */
    module_put(THIS_MODULE);

    printk("Device closed succesfully \n");
    return 0;
}


/*
 * Called when a process, which already opened the dev file, attempts to
 * read from it.
 */
static ssize_t device_read(struct file *filp,    /* see include/linux/fs.h   */
              char *buffer,    /* buffer to fill with data */
              size_t length,    /* length of the buffer     */
              loff_t * offset)
```

```
{
  /*
   * Number of bytes actually written to the buffer
   */
  int bytes_read = 0;

  // Allocating kernel buffer
  int * kernel_buffer = (int*)kmalloc(length * sizeof(int), GFP_KERNEL );

  // Read values from registers into kernel buffer
  kernel_buffer[0] = ioread32(virt_addr);
  kernel_buffer[1] = ioread32(virt_addr+4);
  kernel_buffer[2] = ioread32(virt_addr+8);

  printk("Kernel Buffer[0] = %d \n", kernel_buffer[0]);
  printk("Kernel Buffer[1] = %d \n", kernel_buffer[1]);
  printk("Kernel Buffer[2] = %d \n", kernel_buffer[2]);

  // Using char pointer
  char * kbuf = (char*)kernel_buffer;
  int i ;
  for( i = 0; i < length; i++ )
  {
    // Write / copy values from kernel space to user space buffer
    put_user(*(kbuf++), buffer++); // one char at a time...
    bytes_read++;
  }


  kfree(kernel_buffer);

  /*
   * Most read functions return the number of bytes put into the buffer
   */
  return bytes_read;
}


/*
 * Called when a process writes to dev file: echo "hi" > /dev/hello
 * Next time we'll make this one do something interesting.
 */
static ssize_t device_write(struct file *filp, const char *buff, size_t len, loff_t * off)
{
```

```
  int i;

  char * kernel_buffer = (char*)kmalloc((len+1)*sizeof(char), GFP_KERNEL);

  for( i = 0; i < len; i++ )
  {
      // Get / copy values from user space into kernel buffer
      get_user(kernel_buffer[i], buff+i);
  }

  kernel_buffer[len] = '\0';

  // Using int pointer
  int * int_buf = (int*)kernel_buffer;

  printk(KERN_INFO "Writing %d to register 0 \n", int_buf[0]);
  iowrite32(int_buf[0], virt_addr+0);

  printk(KERN_INFO "Writing %d to register 0 \n", int_buf[1]);
  iowrite32(int_buf[1], virt_addr+4);

  kfree(int_buf);

  /*
   * Again, return the number of input characters used
   */
  return i;
}


/* These define info that can be displayed by modinfo */
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Paul V. Gratz (and others)");
MODULE_DESCRIPTION("Module which creates a character device and allows user
interaction with it");

/* Here we define which functions we want to use for initialization
   and cleanup */
module_init(my_init);
module_exit(my_cleanup);
```